

ARBITER — Mid-Semester Progress Report

Adaptive Multi-Agent System for Iterative Task Generation, Validation and Refinement Gautam Lasgotra (23BCS032) · Chirag Attri (23BCS024) · Aniket Kundal (23BCS015) Guide: Dr. Sonika Gupta, SoCSE · SMVDU Katra · 3 October 2026
Repository: github.com/igautamlasgotra/arbiter

1. Status summary

The core system is **built and working**. An unseen task typed at runtime is answered by a real multi-agent loop: tests are designed, a solution is generated, the solution is executed against those tests, failures are fed back, and the solution is revised until it passes or the budget stops it. The loop, the validators, the budget governor, the baselines and the live web interface are all complete and covered by **39 automated tests that run offline**.

What is **not** done is the measurement. Benchmark results require API keys, which have not yet been provisioned. No results are reported in this document because none have been produced — nothing here is estimated or placeholder.

Area	Status
Data contracts, run state, budget governor	✔ Complete
LLM provider abstraction, response cache, multi-key router	✔ Complete
Sandboxed code execution	✔ Complete
Validators — executable tests, answer matching	✔ Complete
Agent role library — generator, repair, critic	✔ Complete
Test-designer agent (enables unseen runtime tasks)	✔ Complete
Refinement loop with bounded termination	✔ Complete
Baselines A, B, C	✔ Complete
Baseline A+ (budget-matched single agent)	✔ Complete
Adaptive planner (condition D)	✔ Implemented, not yet evaluated
Live web interface with execution trace	✔ Complete
Replay mode (demo safety net)	✔ Complete
Metrics module	✔ Complete
Resumable experiment runner	✔ Complete
Benchmark datasets — code / SQL / math	⌚ Smoke set only; full sets in Phase 2
Experimental results	⌚ Blocked on API keys

2. Verified end-to-end behaviour

The following is a **real, unedited** event stream from the system, produced by running an unseen task through the HTTP interface. It is the exact behaviour described in the project review: one agent produces work, another validates it, feedback returns, the work is revised.

```

event: design_tests  role=test_designer  entry=solve, 2 tests
event: plan          roles=["generator"]
event: generate       role=generator      iteration 1
event: validate       role=python_exec    score=0.50  passed=false
event: decide         refine
event: generate       role=generator      iteration 2
event: validate       role=python_exec    score=1.00  passed=true
event: decide         accept
event: stop           accepted

```

The first attempt was wrong, the executed tests caught it (1 of 2 tests passing → score 0.50), the failure was fed back, and the second attempt passed. Total: 3 LLM calls, 2 iterations.

Two details worth noting to the panel:

- **Validation is not an opinion.** `python_exec` runs the generated program in a sandboxed subprocess and reports which named tests failed. There is no LLM in that path. This matters because Huang et al. (ICLR 2024) showed refinement without external feedback does not help.
- **The score is graded, not binary.** 0.50 means one of two tests passed. Without partial credit the system could not tell "getting closer" from "stuck", and the no-improvement stop condition would be meaningless.

3. What was built

3.1 An agent is a function

No agent framework is used. An agent is a role prompt, one model call, and a typed parse:

```

def generator(task, router, state, feedback=None):
    prompt = build_prompt(task, feedback)
    resp   = router.call(prompt, schema=SOLUTION_SCHEMA, state=state)
    return Solution.model_validate_json(resp.text)

```

Different agents differ only in their system prompt. `repair` is told not to trust the previous approach; `critic` is told to review sceptically. Keeping this explicit — rather than inside LangGraph or AutoGen — is deliberate: the orchestration logic is what the project studies, so it must remain readable.

3.2 The orchestrator is a bounded loop

```

Task → Planner → ┌ Generator → Validators (tools first, critic second) ┐
                  │          ▲                                         │
                  │          feedback ← Decision policy →             │
                  └──────────┘                                         └─┘
                  accept / refine / add agent / stop (budget-governed)

```

The loop can never run unbounded. It stops on acceptance, 5 iterations, 20 LLM calls, 60,000 tokens, 180 seconds, two iterations without improvement, or a repeated identical output — the last being the most common documented multi-agent failure mode (step repetition, 17.1% of failures in the MAST taxonomy, NeurIPS 2025). Every stop reason is recorded.

3.3 The test-designer agent

Benchmark tasks ship with tests. A task typed by a panel member does not. The test-designer agent derives executable tests from the task description and fixes the function signature, which is then given to the generator — otherwise the tester invents `two_sum` while the generator writes `twoSum` and everything fails for the wrong reason.

Generated tests are validated before use: they must define `check_*` functions, must call the declared entry point, and are rejected if they contain imports, file access or other constructs they have no business using.

Stated limitation: model-written tests are not ground truth. They are used only for live unseen tasks. All benchmark measurement uses the dataset's own tests, and the two are never mixed in results.

3.4 Condition A+ — the baseline that makes the project credible

Most student multi-agent projects compare their system against a single LLM call, declare a win, and ignore that the multi-agent system spent five times the tokens. Condition A+ removes that confound: the single agent receives **the same token budget** the adaptive condition actually consumed on that task, spent on repeated sampling with majority voting.

A+ is deliberately denied access to the validator. If it could see test results it would be a refinement loop, not a single agent, and the comparison would be rigged.

3.5 Live interface

The web interface streams agent events over Server-Sent Events as they happen. The panel can type any task and watch the roles, validator verdicts, iterations, LLM calls and token count update live. A replay mode re-streams a stored run with original timings, clearly labelled, so the demonstration survives a dead API key or venue wifi.

ARBITER. Adaptive Role-Based Iterative Task Execution & Refinement no API key — offline only

No API key configured. Copy `.env.example` to `.env` and add `ARBITER_DEMO_KEY` to run live. "Offline demo" works without a key but uses a scripted provider — never use it for results.

Task — type anything, this is not a scripted demo

Write a function that takes a list of integers and returns the second largest distinct value, or None if there isn't one.

Family

code — validated by running tests

Condition

B — generator ↔ validator loop

Run

Offline demo

LIVE METRICS

Iterations —

LLM calls —

Tokens —

Elapsed —

Agents —

Best score —

Validation for code runs the program in a sandboxed subprocess — It is not an LLM opinion. Every step is bounded by an explicit budget.

4. Engineering decisions worth defending in viva

Decision	Alternative considered	Why this
Role selection from a fixed library	Inventing agents at runtime	Runtime invention is unbounded and unreproducible — it cannot be measured, so it would be a demo, not an experiment
Custom ~400-line orchestrator	LangGraph / AutoGen	The orchestration logic <i>is</i> the object of study; AutoGen is also in maintenance mode
Tools validate, LLM critic advises	LLM-only review	Self-correction without external feedback is known to be unreliable; the critic's verdict is logged separately to measure false accepts
Disk response cache from day one	Add caching later	Re-runs cost nothing and become byte-reproducible; without it the free-tier quota makes the experiment infeasible
Graded score, not pass/fail	Binary pass/fail	Partial credit is what lets the loop detect improvement and stop when stuck
No code execution on public host	Sandbox everywhere	Arbitrary remote code execution is not an acceptable risk for a demo

5. Remaining work

Phase	Dates	Work
Phase 2	11 – 31 Oct	Provision API keys; full benchmark datasets (HumanEval+/MBPP+, Spider, GSM8K); evaluate condition D; cross-model validation experiment
Phase 3	1 – 10 Nov	Frozen benchmark sweep, comparative results, ablation on iteration count and validator type, failure classification, deployment, report
Final	23 – 27 Nov	Report, demonstration, viva

Known risks

API quota is the main risk. A full sweep is several thousand model calls against a free tier. Four controls are already implemented: the response cache, round-robin across three keys, a resumable runner that checkpoints after every task and survives quota exhaustion mid-sweep, and a reserved demo key the experiment runner never touches.

Majority voting is weaker for code than for math. Two correct programs rarely match character for character, so A+ is a weaker opponent on code than on math. This is reported as a limitation rather than hidden, and it is itself part of the result.

6. Work distribution

Member	Contribution this phase
Gautam Lasgotra (23BCS032)	System architecture, orchestration loop and decision policy, budget governor, provider abstraction with caching and key rotation, sandboxed execution, web interface and streaming, integration
Chirag Attri (23BCS024)	Agent role library and prompt design, structured output schemas, test-designer agent, LLM critic, provider integration
Aniket Kundal (23BCS015)	Benchmark task format and smoke dataset, execution validators, metrics module, resumable experiment runner, test suite

Verification: `pytest -q` → 39 passed, offline, no API key required. **Run the demo:** `python -m uvicorn
arbiter.web.app:app --reload` → `http://127.0.0.1:8000`